CSE 4105 · SYSTEM PROGRAMMING · SHORT COURSE

Class 3

Assemblers

two passes, the symbol table, the object program, relocation

CLASS

3 / 10

20 slides, taken from the full course:
Lecture 3.1, Lecture 3.2

Md Mahbub E Noor
Assistant Professor, Department of CSE
University of Barishal

The five jobs of an assembler

1. Mnemonic → opcode

`LDA` becomes `00`. Like looking up a word in a dictionary. The dictionary is **OPTAB**.

2. Label → address

`ZERO` becomes `2033`. The assembler must remember every label's address: **SYMTAB**.

3. Build the instruction

Join opcode, x bit and address in the right format: `04` + `2033` = `042033`.

4. Constants → bytes

`BYTE C'BANANA'` becomes `42 41 4E 41 4E 41`; `WORD 65` becomes `000041`.

5. Write the output

The **object program** for the loader, and a **listing** for humans.

C link: `gcc -c` does the same five jobs for x86-64 when it runs `as` on your .s file (Lecture 1).

Book: Beck §2.1

Our example program: count the A's in BANANA

C version

```c
char text[6] = "BANANA";
int total = 0;

for (i = 0; i < 6; i++)
    if (text[i] == 'A')
        total++;
```

DIFFERENT FROM C

Data after code

In C you declare variables before using them. Here the data words come at the end, after the code that uses them.

count.asm (SIC)

```
COUNT    START   2000
FIRST    LDX     ZERO
LOOP     LDA     ZERO
         LDCH    TEXT,X
         COMP    LETTERA
         JEQ     FOUND
NEXT     TIX     LENGTH
         JLT     LOOP
         J       DONE
FOUND    LDA     TOTAL
         ADD     ONE
         STA     TOTAL
         J       NEXT
DONE     J       DONE
TEXT     BYTE    C'BANANA'
LENGTH   WORD    6
LETTERA  WORD    65
ZERO     WORD    0
TOTAL    WORD    0
BUFFER   RESB    100
ONE      WORD    1
         END     FIRST
```

What it does

X steps through the six bytes of TEXT. Each byte is compared with LETTERA (65 = 'A'). On a match, the program jumps to FOUND and adds ONE to TOTAL.

We ran it: TOTAL = 3, after 51 instructions (the 51st is J DONE, the stop loop).

The problem: forward references

```
2000 FIRST   LDX   ZERO      forward: ZERO is defined at 2033
2006         LDCH  TEXT,X    forward: TEXT is defined at 2027
200C         JEQ   FOUND     forward: FOUND is defined at 2018
2012         JLT   LOOP      backward: LOOP was defined at 2003
2018 FOUND   LDA   TOTAL     forward: TOTAL is defined at 2036
```

10 of 13

instructions in COUNT use a label **before** it is defined. When the assembler reaches line 2000, it has no idea where ZERO is.

SAME AS C

C has the same problem

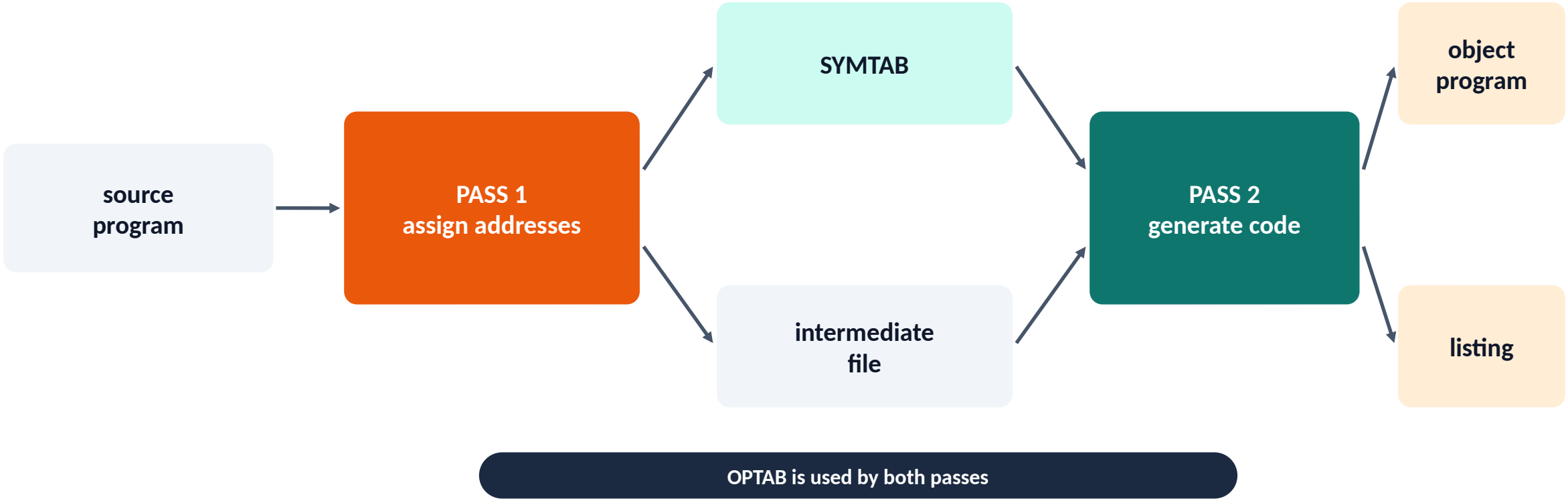
Call `counta()` before its definition and gcc warns: implicit declaration. A name must be known before it is used.

DIFFERENT FROM C

But a different solution

C makes **you** declare first (a prototype, or extern). The assembler instead **reads the program twice**: once to learn the addresses, once to use them.

The solution: read the program twice



Analogy: making the index of a book. First you read the whole book and write down the page of every topic. Only then can you print "see page 47" anywhere, even on page 3.

Book: Beck §2.1.2 (assembler algorithm and data structures)

Three data structures, and how they look in C

OPTAB *the opcode table*

Fixed: the same for every program. Mnemonic → opcode (and size). Usually a hash table built into the assembler.

sicasm.c

```
struct op {
  const char *name;
  int code;
};
const struct op OPTAB[] =
 {{"ADD",0x18}, {"LDA",0x00}, …};
```

SYMTAB *the symbol table*

Grows during Pass 1, read in Pass 2. Label → address. A hash table, because it is searched for every operand.

sicasm.c

```
struct sym {
  char name[16];
  int  addr;
};
struct sym SYMTAB[200];
int nsyms = 0;
```

LOCCTR *the location counter*

One integer: the address the current line will get. Starts at the START operand, grows by each line's size.

sicasm.c

```
int locctr;

locctr = strtol(operand,
                NULL, 16);
…
locctr += 3;
```

In a real assembler OPTAB and SYMTAB are hash tables. In our lab, a simple array and a for-loop search are fine: SIC programs are small.

Book: Beck §2.1.2

The Pass 1 algorithm, in five steps

1. Read the first line. If the opcode is **START**, set LOCCTR to its operand (a hex number). Otherwise LOCCTR = 0.
2. For each line until **END**: if it has a **label**, search SYMTAB. Found → error "duplicate symbol". Not found → insert (label, LOCCTR).
3. Look at the **opcode** to find the size: an instruction or WORD = 3, RESW n = 3n, RESB n = n, BYTE = length of the constant. Unknown → error.
4. Write the line, with its LOCCTR, to the intermediate file. Then LOCCTR += size.
5. At **END**: program length = LOCCTR − starting address.

the same in C (sicasm.c)

```
while (fgets(buf, …)) {
  split(buf, ln);
  ln->loc = locctr;
  if (ln->label[0])
    add_symbol(…);
  if (is_instr(op))
    locctr += 3;
  else if (WORD) …
  else if (RESW) …
  else if (RESB) …
  else if (BYTE) …
}
length = locctr - start;
```

Book: Beck §2.1.2, Pass 1 algorithm

How many bytes does each line take?

Line	Size	Example from COUNT	LOCCTR before → after
any instruction	3	`FIRST  LDX  ZERO`	`2000 → 2003`
WORD	3	`LENGTH WORD 6`	`202D → 2030`
RESW n	3 × n	`TABLE RESW 10`	`+ 30 = + 1E`
RESB n	n	`BUFFER RESB 100`	`2039 → 209D`
BYTE C'…'	1 per character	`TEXT BYTE C'BANANA'`	`2027 → 202D`
BYTE X'…'	1 per 2 hex digits	`INPUT BYTE X'F1'`	`+ 1`
START, END	0	`COUNT START 2000`	`LOCCTR ← 2000`

Trap: decimal vs hex

The operand of RESB, RESW and WORD is **decimal**. LOCCTR is **hex**.

`RESB 100` adds 100 = **64 hex**:
`2039 + 64 = 209D`

Not 2039 + 100 = 2139!

Program length = 20A0 − 2000 = **A0** hex = 160 bytes.

Book: Beck §2.1.2

Pass 1 on COUNT: every line gets an address

LOC	label	op	operand
	COUNT	START	2000
2000	FIRST	LDX	ZERO
2003	LOOP	LDA	ZERO
2006		LDCH	TEXT,X
2009		COMP	LETTERA
200C		JEQ	FOUND
200F	NEXT	TIX	LENGTH
2012		JLT	LOOP
2015		J	DONE
2018	FOUND	LDA	TOTAL
201B		ADD	ONE

LOC	label	op	operand
201E		STA	TOTAL
2021		J	NEXT
2024	DONE	J	DONE
2027	TEXT	BYTE	C'BANANA'
202D	LENGTH	WORD	6
2030	LETTERA	WORD	65
2033	ZERO	WORD	0
2036	TOTAL	WORD	0
2039	BUFFER	RESB	100
209D	ONE	WORD	1

SYMTAB	address
FIRST	2000
LOOP	2003
NEXT	200F
FOUND	2018
DONE	2024
TEXT	2027
LENGTH	202D
LETTERA	2030
ZERO	2033
TOTAL	2036
BUFFER	2039
ONE	209D

length

20A0
− 2000
= 00A0

12 labels, all known at the end of Pass 1.

The Pass 2 algorithm, in six steps

1. Read each line of the intermediate file. Its LOCCTR is already there from Pass 1.
2. **Instruction:** opcode from OPTAB. Operand from SYMTAB; not there → error "undefined symbol". If it ends in `,X`, set x = 1.
3. Assemble: **opcode** (8 bits) | **x** (1 bit) | **address** (15 bits) → 6 hex digits.
4. **BYTE / WORD:** convert the constant to hex. **RESB / RESW:** no code, but end the current text record.
5. Add the code to the current **T record**. If it would pass 30 bytes, write the record and start a new one.
6. At **END**: write the last T record, then the **E record** with the first instruction's address.

the same in C (sicasm.c)

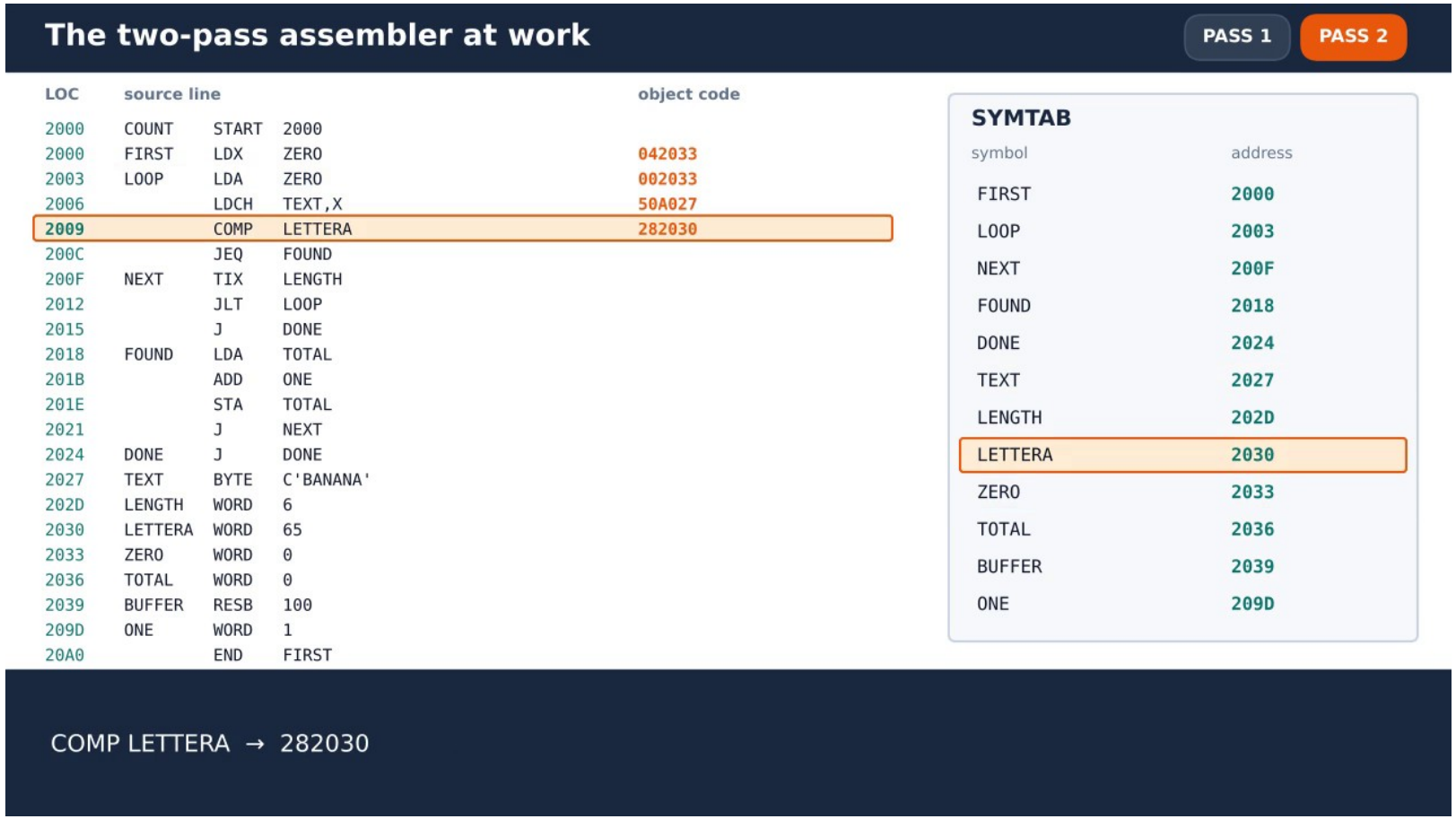
```
for (i = 0; i < nlines; i++) {
  opc = find_op(L[i].opcode);
  if (opc >= 0) {
    addr = find_sym(name);
    if (addr < 0) error…
    sprintf(objcode,
      "%02X%04X", opc,
      (x << 15) | addr);
  } else if (WORD) …
  else if (BYTE) …
}
```

Book: Beck §2.1.2, Pass 2 algorithm

Pass 2 on COUNT: every line gets its code

LOC	source			object code	how
2000	FIRST	LDX	ZERO	**042033**	LDX = 04, ZERO = 2033
2003	LOOP	LDA	ZERO	**002033**	LDA = 00, ZERO = 2033
2006		LDCH	TEXT,X	**50A027**	LDCH = 50, TEXT = 2027 + 8000 (x=1)
2009		COMP	LETTERA	**282030**	COMP = 28, LETTERA = 2030
200C		JEQ	FOUND	**302018**	JEQ = 30, FOUND = 2018
200F	NEXT	TIX	LENGTH	**2C202D**	TIX = 2C, LENGTH = 202D
2012		JLT	LOOP	**382003**	JLT = 38, LOOP = 2003
2015		J	DONE	**3C2024**	J = 3C, DONE = 2024
2018	FOUND	LDA	TOTAL	**002036**	LDA = 00, TOTAL = 2036
201B		ADD	ONE	**18209D**	ADD = 18, ONE = 209D
201E		STA	TOTAL	**0C2036**	STA = 0C, TOTAL = 2036
2021		J	NEXT	**3C200F**	J = 3C, NEXT = 200F
2024	DONE	J	DONE	**3C2024**	J = 3C, DONE = 2024
2027	TEXT	BYTE	C'BANANA'	**42414E414E41**	ASCII: B=42 A=41 N=4E …
202D	LENGTH	WORD	6	**000006**	decimal 6 = hex 000006
2030	LETTERA	WORD	65	**000041**	decimal 65 = hex 000041
2033	ZERO	WORD	0	**000000**	decimal 0 = hex 000000
2036	TOTAL	WORD	0	**000000**	decimal 0 = hex 000000
2039	BUFFER	RESB	100		no code (only reserves 100 bytes)
209D	ONE	WORD	1	**000001**	decimal 1 = hex 000001

Watch both passes run



Click the video to play or pause. About 1½ minutes, no sound.

Watch for

1. In **Pass 1** only the LOC column and SYMTAB fill up. No code yet.
2. At **JEQ FOUND**, Pass 1 just counts 3 bytes and moves on.
3. In **Pass 2** the code column fills, using the finished SYMTAB.
4. **RESB** gets an address but no code.
5. The last frame: the **H, T, E** records.

Three record types: Header, Text, End

H Header

one per program



columns: 1 · 2–7 · 8–13 · 14–19

Tells the loader the program's name, where it starts and how much memory it needs.

T Text

as many as needed



columns: 1 · 2–7 · 8–9 · 10–69

Bytes to put into memory, starting at the given address.

E End

one per program



columns: 1 · 2–7

Where to jump when loading is done: the operand of END.

Book: Beck §2.1.1 (the simple object program format)

The object program of COUNT, field by field

```
H COUNT  002000 0000A0
T 002000 1E 042033002033 50A027282030 20182C202D38 20033C202400 2036 18209D
T 00201E 1B 0C20363C200F 3C2024424 14E414E41000006000041000000000
T 00209D 03 000001
E 002000
```

H: Name COUNT (padded to 6), start 002000, length 0000A0 = 160 bytes.

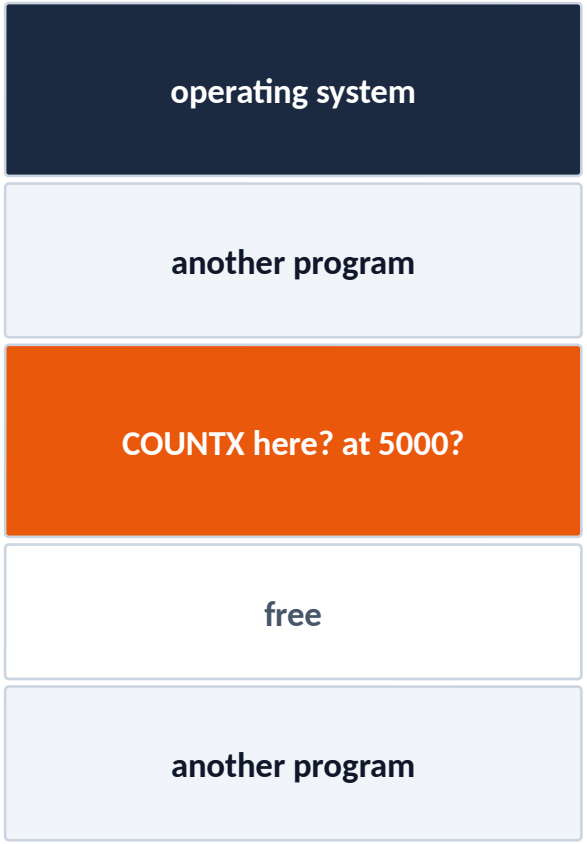
T 1: Starts at 002000, 1E = **30 bytes**: full, so the assembler starts a new record.

T 2: Starts at 00201E, 1B = **27 bytes**, from STA TOTAL to the last WORD. It stops at the RESB.

T 3: Starts at 00209D, **after the 100-byte gap**: just ONE WORD 1.

E: Start execution at 002000 = FIRST, the operand of END.

Why must a program work at any address?



memory while several programs run

With several programs in memory at once (multiprogramming), the **loader** picks a free place each time. The assembler, which ran long before, cannot know where.

So `START 0` means: "assemble as if at 0, but be ready to move". The program must still work when its first byte lands at 5000.

where.c run twice on the lab PC

```
$ ./where
main    is at 0x55c51af75149
counter is at 0x55c51af78010
$ ./where
main    is at 0x559168e93149
counter is at 0x559168e96010
```

SAME AS C

Your C programs move too

Linux loads every run at a new address (ASLR). But the **distance** from main to counter is always 0x2ec7: relative addresses survive the move.

Which instructions break when the program moves?

Kind of operand	What is stored	If the program moves 5000 up	Needs fixing?
PC-relative	a distance	the distance is the same	no
base-relative	a distance from B	B is loaded at run time with the moved address	no
immediate constant	a number	a number is a number	no
format 4 with a symbol	an absolute address	points to the old place	YES
SIC direct address	an absolute address	points to the old place	YES

This is one big reason SIC/XE has relative addressing: programs become easy to move.

2 of 19

instructions in COUNTX need fixing.

13 of 13

in the SIC program COUNT: every address in plain SIC is absolute.

Book: Beck §2.2.2 (program relocation)

The Modification (M) record



columns: 1 · 2–7 (address of the field, relative to the program start) · 8–9 (length of the field in half-bytes)

It tells the loader: "after loading, **add the program's start address** to this field".

Example: +JSUB REPORT at 0022 → 4B101044



the 20-bit address field = the last 5 half-bytes

Rule for format 4: M record address = instruction address + 1, length 05.

`M00002305` says:

- the field starts in byte **0023** (the byte after the opcode),
- it is **05** half-bytes long (20 bits).

The field starts in the middle of byte 0023, so the length is counted in half-bytes, not bytes.

Book: Beck §2.2.2

Watch the loader fix the program

Relocation: one program, two load addresses

```
object program countx.obj

HCOUNTX000000001047
T0000001C17203269101041B410750006B40053A02A2…
T00001C190320190F40004B1010443E200C03200C190…
T000038090000042414E414E41
T001044034F0000
M00000405
M00002305
E000000
```

instruction		kind
STL	RETADR	PC-relative
+LDB	#TOTAL	format 4
JLT	LOOP	PC-relative
STA	TOTAL	base-relative
+JSUB	REPORT	format 4
J	@RETADR	PC-relative

loaded at 0000

run it: TOTAL = 3

address	bytes in memory
0000	172032
0003	69101041
0019	3B2FF0
001F	0F4000
0022	4B101044
0026	3E200C

loaded at 5000

address	bytes in memory
5000	172032
5003	69106041
5019	3B2FF0
501F	0F4000
5022	4B106044
5026	3E200C

And the others? STL, JLT, J @RETADR are PC-relative; STA TOTAL is base-relative. Their bytes are the same at 0000 and 5000: a distance does not change when the whole program moves.

Click the video to play or pause. About 50 seconds, no sound.

Watch for

1. Loaded at **0000**, nothing needs fixing.
2. Loaded at **5000**, the two red rows still hold old addresses.
3. Each **M record** adds 5000 to one field.
4. The PC-relative rows have the **same bytes** at both addresses.
5. Both runs give **TOTAL = 3**.

Key ideas from Class 3

1. An assembler translates mnemonics to opcodes and symbols to addresses, and writes an object program.
2. Forward references are why it reads the source twice.
3. Pass 1 gives every line an address (LOCCTR) and fills SYMTAB; Pass 2 writes the object code.
4. The object program has H (header), T (text) and E (end) records.
5. M records list the address fields the loader must change when the program is loaded somewhere else.

Homework and practice

Homework 3 (bring it to the next class)

1. Why can a one-pass assembler not translate JEQ FOUND when FOUND comes later? How does two-pass solve it?
2. Give the LOCCTR after each line of a 4-line program: a 3-byte instruction, WORD 5, RESB 10, RESW 2 (start at 1000).
3. Read the H, T and E records of COUNT: what are the start address, the length and the first instruction?

Try it in the browser

Two-pass assembler: pass 1 one line at a time

`labs/lab03.html`

On the course website: open the lab page and choose "Try it here".

Next class: Macros and Loaders